

Estado do projeto — onde as duas branches estão

Levantamento de 6 de outubro de 2026. Compara `main` e `feature/app`, lista o que já está pronto e o que falta. Para o contrato tela a tela, veja `docs/design/telas.md`; para a lista de rotas, o `README`.

1. As duas branches

Repositório: `git@github.com:DaviMaiaF/Decada-store.git`.

REFERÊNCIA	COMMIT	SITUAÇÃO
<code>origin/main</code>	<code>a9ce1fe</code> (merge do PR #3)	É a linha de verdade. Contém tudo
<code>main</code> (local)	<code>a9ce1fe</code>	Em dia
<code>origin/feature/app</code>	<code>2e54197</code>	Contida na <code>main</code> ; nada exclusivo
<code>feature/app</code> (local)	<code>a9ce1fe</code>	Em dia com a <code>main</code>

As duas branches estão no mesmo ponto. O PR #3 foi mesclado, e os dois commits que estavam só na `feature/app` passaram a ser a linha de verdade do projeto. Não há nada pendente de integração.

O que entrou pelo PR #3

Dois commits, 20 arquivos, +1.413 / -154 linhas:

COMMIT	O QUE ENTREGA
<code>ccb1bbd</code>	feat: item avulso na lista de compras — POST / DELETE em <code>/shopping-lists/{id}/items</code> , migração <code>45f461c0d4c0</code> , <code>plan_item_id</code> nulável, busca em <code>/products/search</code> e o cartão "fora da prescrição" no Mercado
<code>2e54197</code>	feat: acrescentar à compra o que a receita pede e o que acabou em casa — o botão "+ ingrediente" da Despensa e o seletor de unidade compartilhado entre despensa e item avulso

Os dois vieram com teste (`test_shopping.py`, `test_api.py`, `MarketScreen.test.tsx`, `PantryScreen.test.tsx`) e já estavam descritos em `telas.md`.

2. Verificação feita hoje

Rodado nesta máquina, com o Postgres do `docker compose` no ar, na ponta da `feature/app`:

CHECAGEM	RESULTADO
<code>ruff check app tests</code>	limpo
<code>pytest (backend)</code>	358 testes, todos passando
<code>tsc --noEmit (mobile)</code>	limpo
<code>jest (mobile)</code>	149 testes em 11 suítes, todos passando

Um ponto a vigiar: na primeira execução, com a máquina ocupada, 5 testes do app caíram por tempo (`AppNavigator.test.tsx` levou 24,7 s contra 5,2 s na segunda rodada, que passou inteira). Não é erro de código — é prazo de espera apertado em teste assíncrono. Se o CI começar a piscar vermelho sem mudança de código, é aqui que se olha.

3. O que já temos

Backend — completo para o MVP

Onze tabelas, 21 rotas, regra de negócio isolada em `app/services/` e testada:

SERVIÇO	O QUE RESOLVE
<code>text.py</code> · <code>units.py</code> · <code>parsing.py</code>	normalização, conversão de grandeza e leitura da linha do plano
<code>matching.py</code>	casamento item do plano ↔ produto, com score — confirmação sempre humana
<code>pricing.py</code>	média, mediana e desvio por região , com data e origem; arredondamento de embalagem
<code>pantry.py</code>	quanto da compra a despensa cobre, com abatimento parcial
<code>shopping.py</code>	geração da lista, desconto da despensa, item avulso
<code>simulation.py</code>	custo com outras quantidades, sem gravar nada
<code>recipes.py</code>	disponibilidade da receita contando despensa + lista
<code>pdf.py</code> · <code>import_plan.py</code>	PDF com texto → plano, com consentimento obrigatório e relatório do descarte
<code>security.py</code>	bcrypt e JWT

App mobile — a jornada principal inteira

Login, envio do PDF com aceite do termo, confirmação dos produtos, escolha da região, lista de compras por corredor, despensa com quantidade, receitas por disponibilidade, aba Economia e

a trilha **Enviar** → **Confirmar** → **Região** → **Comprar** por cima das abas. O app reencontra plano e lista ao ser reaberto, consultando o servidor — não guarda ponteiro para dado de saúde no aparelho.

Documentação e infraestrutura

- `docs/modelo-dados.md` (ER e regras de exclusão), `docs/lgpd.md` (cada exigência apontando o teste que a sustenta), `docs/design/` (protótipos, sistema de design, contrato das telas), `docs/demo/` (roteiro de 5 minutos e dossiê técnico), `docs/plano-de-trabalho.pdf`.
- CI em `.github/workflows/ci.yml`: pytest e jest em todo PR e em todo push na `main`, com Postgres de verdade e uma etapa que falha se o banco não responder — sem ela, banco fora do ar viraria CI verde com zero teste executado.

4. O que precisamos fazer

4.1 Integrar o que estava pronto — concluído

PASSO	SITUAÇÃO
<code>git push origin feature/app</code>	 feito
Abrir o <code>PR #3</code>	 aberto
CI verde nos dois trabalhos	 jest em 43 s, pytest em 1 min 17 s
Mesclar na <code>main</code> e atualizar a <code>main</code> local	 mesclado em <code>a9ce1fe</code>

Duas funcionalidades testadas e documentadas estavam fora da linha de verdade do projeto por nada. Agora não estão mais.

4.2 Dentro do MVP e ainda não feito

O QUE	ONDE	ESFORÇO
Nome e CRN da nutricionista	<code>MealPlan</code> tem <code>nutritionist_name</code> , e <code>api.ts</code> já sabe enviá-lo — mas a tela de upload não tem o campo, e CRN não existe no modelo . Pede migração, campo no schema, dois campos na tela	pequeno
Card de custo na aba Dieta	<code>telas.md</code> §2 prevê o card "~ R\$ X" com atalho para o Mercado na confirmação. O atalho existe; o custo não aparece ali	pequeno

Os dois são pequenos. O que de fato falta construir é a **despesa por foto**, que ganhou seção própria logo abaixo por ser funcionalidade nova, e não ajuste de tela. Fora esses três, o que `telas.md` marca como MVP está  e consumido pelo app.

4.3 A próxima funcionalidade: encher a despensa por foto

A pessoa fotografa o que tem em casa — a prateleira, a geladeira, as compras em cima da mesa — e **o app diz o que viu**. O que ela confirmar vira item da despensa.

É o caminho mais curto para o problema que a despensa tem hoje: cadastrar item por item, digitando e escolhendo o produto, é trabalhoso o bastante para a pessoa simplesmente não fazer — e despensa vazia desliga as duas coisas que dependem dela, o abatimento da lista de compras e a ordem das receitas.

O reconhecimento propõe; quem decide é a pessoa. A foto devolve nomes — "arroz", "ovos", "leite" —, não produtos do catálogo nem quantidades. Então o resultado da foto é uma *proposta*, confirmada na tela, exatamente como já acontece com o casamento do plano alimentar: nunca 100% automático. Item que o reconhecimento não souber resolver aparece como não identificado e espera a pessoa, do mesmo jeito que o import de PDF já faz com as linhas que descartou. Nada entra na despensa sozinho.

Metade do caminho já existe e tem teste. Transformar texto livre em produto do catálogo com score é o que `services/text.py` e `services/matching.py` fazem desde a etapa 5, e é o mesmo motor que `GET /products/search` usa no cadastro manual. O que falta é só a ponta nova: imagem → lista de nomes. Daí para a frente, o caminho é o que já está construído e coberto.

PEÇA	SITUAÇÃO
imagem → nomes de alimentos	a construir — é a única peça realmente nova
nomes → produtos candidatos, com score	✓ <code>matching.py</code> , testado
candidatos → <code>PantryItem</code> com produto e quantidade	✓ <code>POST /pantry</code> e <code>PATCH /pantry/{id}</code> , testados
tela de confirmação dos itens	a construir — reaproveita o seletor de unidade e os chips que a Despensa já tem
câmera no app	a construir — <code>expo-image-picker</code> , com câmera e galeria

A quantidade continua sendo pergunta. Foto não pesa arroz. O item reconhecido entra já vinculado ao produto, com a medida em branco ou na embalagem padrão — que é o comportamento que a despensa já tem hoje. Vale lembrar a regra que isso aciona: item sem quantidade **conta para a receita, mas não abate da lista de compras**. Ou seja, a foto melhora a sugestão de receitas no mesmo instante, e só ajuda na compra depois que a pessoa disser quanto tem.

A imagem não é guardada. Foto do interior da casa de alguém é dado pessoal, e a decisão 5 pede minimização: o app extrai os itens e descarta o arquivo — não há coluna para imagem, nem bucket, nem nada a apagar depois. Se o reconhecimento for feito por um serviço de

terceiros, isso precisa estar no termo de consentimento antes de a primeira foto sair do aparelho.

O que se testa é a nossa regra, não o modelo. Acerto de reconhecimento não é teste de unidade — varia com luz, ângulo e sorte. O teste cobre a tradução: dada uma lista de nomes devolvida pelo reconhecimento (dublado), saem os candidatos certos, o não identificado é sinalizado, e o `PantryItem` nasce com produto e grandeza compatíveis. É a mesma forma do `test_matching.py`.

Isto não reabre a decisão 7. O que está fora do MVP ali é OCR de *plano alimentar* — ler a prescrição de uma foto, onde errar significa comida sumindo da dieta de alguém. Reconhecer comida na despensa é outro risco: errar custa um item a mais ou a menos numa lista que a pessoa está olhando. Vale deixar a decisão 7 mais precisa quando esta entrar.

4.4 NFC-e: continua no plano, continua fora do MVP

A decisão 3 segue de pé — **a fonte principal de preço é o QR Code da NFC-e** —, e a foto da despensa não a substitui: uma enche a despensa, a outra formaria o preço. Hoje todo preço vem do seed fictício (`is_fictitious = true`, origem `seed`), e o app chega a oferecer "escanear a NFC-e" como saída quando a região não tem coleta — uma saída que ainda não leva a lugar nenhum.

Quando entrar, entram juntos: leitor de QR Code no app, parser do retorno da SEFAZ, casamento dos itens da nota com o catálogo e `PriceRecord` com origem `nfce` — **com teste**, que o CLAUDE.md exige nominalmente para o parser de NFC-e.

4.5 Fora do MVP, por decisão registrada

Refeições com horário e status (depende de um modelo `Meal`), hidratação, carrossel semanal, OCR do plano alimentar por foto, projeção mensal, métricas de desperdício, validade de item da despensa, exportação para WhatsApp, fotos de receita e dicas de substituição geradas pelo app — esta última proibida pela decisão 4, não adiada.

5. Resumo

O projeto está em beta funcional: backend completo, app cobrindo a jornada inteira, 507 testes passando entre as duas suítes, CI de pé e documentação em dia. As duas branches não brigam — `feature/app` só está na frente.

Com o PR #3 mesclado, o que resta até "entregável" são **os dois campos da nutricionista** (§4.2), ambos pequenos. Depois deles, a próxima funcionalidade é a **despensa por foto** (§4.3) — menor do que parece, porque a parte difícil, que é virar produto do catálogo a partir de um nome solto, já está pronta e testada desde a etapa 5.

A direção da foto já está propagada para o `CLAUDE.md` (decisão 11), o `README.md` e o `docs/design/telas.md` (seção 4), então quem pegar o projeto depois não precisa deste documento para saber dela.